# trabalho2_experimentos

February 27, 2026

# 1 Experimentos do Trabalho 2 - Reconhecimento de Padrões

- Aluno: Lucas José Lemos Braz

### 1.0.1 1. Imports e Configurações

[6]:
```
import time
import warnings
import os
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats

# Adicionando o diretório raiz ao path para importar os modelos
sys.path.append(os.path.abspath("../../.."))
from classificao_padroes.models_trabalho2 import mcovar1, mcovar2, mcovar3,␣
↪mcovar4

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid")
```

### 1.0.2 2. Carregamento dos Dados

Nesta seção, carregamos os datasets do robô Wall-Following. - `sensor_readings_24.data`: 24 sensores de ultrassom. - `sensor_readings_4.data`: 4 sensores de ultrassom simplificados.

[7]:
```
# Caminhos relativos para os arquivos de dados
data_path_24 = "../../data/interin/wall+following+robot+navigation+data/
↪sensor_readings_24.data"
data_path_4 = "../../data/interin/wall+following+robot+navigation+data/
↪sensor_readings_4.data"

# Nomes das colunas
col_names_24 = [f"US{i+1}" for i in range(24)] + ["Class"]
```

```
col_names_4 = ["SD_front", "SD_left", "SD_right", "SD_back", "Class"]

# Carregando os datasets
data_24_df = pd.read_csv(data_path_24, names=col_names_24)
data_4_df = pd.read_csv(data_path_4, names=col_names_4)

# Separando features (X)
X_24 = data_24_df.drop("Class", axis=1).values
X_4 = data_4_df.drop("Class", axis=1).values

print(f"Dataset 24 sensores: {data_24_df.shape}")
print(f"Dataset 4 sensores: {data_4_df.shape}")
```

```
Dataset 24 sensores: (5456, 25)
Dataset 4 sensores: (5456, 5)
```

### 1.0.3 3. Funções Auxiliares

Definimos as funções para cálculo de covariância, norma e utilitários para plotagem e benchmarking. Agora incluímos também os métodos `mcovar1`, `mcovar2` e `mcovar4`.

[8]:
```
def calc_cov_matrix(data):
    """Calcula a matriz de covariância (variáveis nas colunas) usando NumPy."""
    return np.cov(data, rowvar=False, bias=True)

def calc_norm_diff(cov_np: np.ndarray, data: np.ndarray):
    """Calcula a norma entre o a matriz de covariância e a matriz de␣
 ↪covariância calculada pelo numpy."""
    return np.linalg.norm(cov_np - data)

def plot_covariance_heatmap(data, col_labels, title):
    """Plota o heatmap da matriz de covariância."""
    cov_matrix = calc_cov_matrix(data)
    plt.figure(figsize=(12, 10))
    sns.heatmap(cov_matrix, annot=True, cmap="coolwarm", fmt=".1f",␣
 ↪square=True,cbar=False,
                xticklabels=col_labels, yticklabels=col_labels)
    plt.title(title)
    plt.show()

def run_benchmark(data, n_rounds=100):
    """Executa o benchmark e retorna listas de tempos."""
    methods = ['mcovar1', 'mcovar2', 'mcovar3', 'mcovar4', 'NumPy Cov']
    results = {m: [] for m in methods}

    # Dados transpostos para os métodos mcovar (esperam (p, N))
    data_T = data.T
```

```
    for _ in range(n_rounds):

        # mcovar1 (Loop)
        start = time.perf_counter()
        mcovar1(data_T)
        results['mcovar1'].append(time.perf_counter() - start)

        # mcovar2 (Matrix Ops)
        start = time.perf_counter()
        mcovar2(data_T)
        results['mcovar2'].append(time.perf_counter() - start)

        # mcovar3 (Outer Product)
        start = time.perf_counter()
        mcovar3(data_T)
        results['mcovar3'].append(time.perf_counter() - start)

        # mcovar4 (Outer Product)
        start = time.perf_counter()
        mcovar4(data_T)
        results['mcovar4'].append(time.perf_counter() - start)

        # NumPy Covariance
        start = time.perf_counter()
        calc_cov_matrix(data)
        results['NumPy Cov'].append(time.perf_counter() - start)

    return results
```

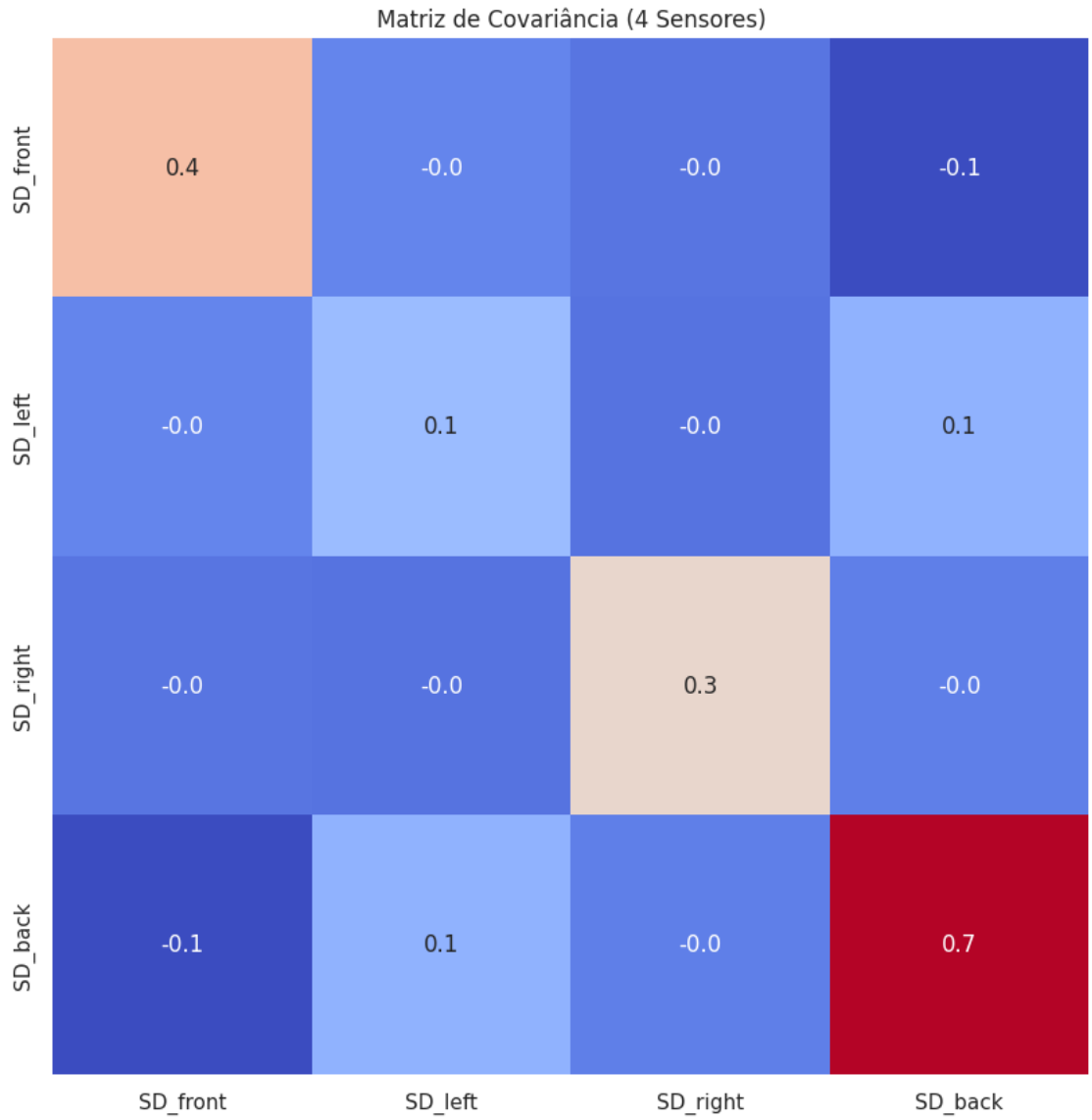
### 1.0.4 4. Análise da Matriz de Covariância

Visualização da matriz de covariância para ambos os casos.

[9]:
```
# Visualização da Matriz de Covariância para 24 Sensores
plot_covariance_heatmap(X_24, col_names_24[:-1], "Matriz de Covariância (24␣
↪Sensores)")

# Visualização da Matriz de Covariância para 4 Sensores
plot_covariance_heatmap(X_4, col_names_4[:-1], "Matriz de Covariância (4␣
↪Sensores)")
```

Matriz de Covariância (24 Sensores)

| | US1 | US2 | US3 | US4 | US5 | US6 | US7 | US8 | US9 | US10 | US11 | US12 | US13 | US14 | US15 | US16 | US17 | US18 | US19 | US20 | US21 | US22 | US23 | US24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| US1 | 0.6 | 0.3 | 0.1 | 0.1 | 0.0 | -0.1 | -0.3 | -0.2 | -0.3 | -0.3 | -0.2 | -0.2 | -0.2 | -0.3 | -0.2 | 0.1 | 0.1 | 0.1 | 0.2 | 0.2 | 0.2 | 0.4 | 0.4 | 0.4 |
| US2 | 0.3 | 2.0 | 0.6 | 0.3 | 0.3 | 0.1 | -0.1 | -0.0 | -0.1 | -0.2 | -0.4 | -0.3 | -0.4 | -0.3 | -0.1 | -0.0 | -0.1 | -0.0 | 0.0 | 0.1 | 0.1 | -0.0 | 0.1 | 0.2 |
| US3 | 0.1 | 0.6 | 1.6 | 0.6 | 0.6 | 0.3 | -0.1 | -0.3 | -0.4 | -0.3 | -0.3 | -0.4 | -0.3 | -0.3 | -0.3 | -0.0 | -0.1 | -0.1 | -0.1 | -0.0 | -0.0 | 0.3 | 0.2 | 0.1 |
| US4 | 0.1 | 0.3 | 0.6 | 1.7 | 1.1 | 0.5 | -0.1 | -0.2 | -0.3 | -0.2 | -0.2 | -0.3 | -0.3 | -0.6 | -0.6 | 0.0 | -0.1 | -0.2 | -0.3 | -0.1 | -0.1 | 0.6 | 0.3 | 0.1 |
| US5 | 0.0 | 0.3 | 0.6 | 1.1 | 1.8 | 0.8 | 0.1 | -0.0 | -0.3 | -0.2 | -0.2 | -0.2 | -0.2 | -0.6 | -0.7 | -0.1 | -0.2 | -0.3 | -0.4 | -0.3 | -0.2 | 0.2 | 0.2 | 0.1 |
| US6 | -0.1 | 0.1 | 0.3 | 0.5 | 0.8 | 1.6 | 0.7 | 0.3 | 0.2 | -0.0 | -0.3 | -0.3 | -0.2 | -0.3 | -0.4 | -0.2 | -0.3 | -0.3 | -0.4 | -0.3 | -0.2 | -0.2 | -0.2 | -0.1 |
| US7 | -0.3 | -0.1 | -0.1 | -0.1 | 0.1 | 0.7 | 2.0 | 0.6 | 0.8 | 0.5 | 0.2 | 0.1 | 0.0 | 0.2 | 0.2 | -0.2 | -0.3 | -0.3 | -0.4 | -0.4 | -0.3 | -0.8 | -0.4 | -0.4 |
| US8 | -0.2 | -0.0 | -0.3 | -0.2 | -0.0 | 0.3 | 0.6 | 1.2 | 0.8 | 0.8 | 0.4 | 0.0 | -0.0 | -0.0 | -0.1 | -0.2 | -0.2 | -0.2 | -0.3 | -0.3 | -0.3 | -0.5 | -0.2 | -0.2 |
| US9 | -0.3 | -0.1 | -0.4 | -0.3 | -0.3 | 0.2 | 0.8 | 0.8 | 1.8 | 1.0 | 0.5 | -0.0 | 0.0 | 0.1 | 0.3 | -0.2 | -0.3 | -0.3 | -0.4 | -0.2 | -0.2 | -0.6 | -0.5 | -0.5 |
| US10 | -0.3 | -0.2 | -0.3 | -0.2 | -0.2 | -0.0 | 0.5 | 0.8 | 1.0 | 1.7 | 0.8 | 0.2 | 0.1 | -0.0 | 0.1 | -0.0 | -0.2 | -0.3 | -0.4 | -0.3 | -0.2 | -0.4 | -0.4 | -0.4 |
| US11 | -0.2 | -0.4 | -0.3 | -0.2 | -0.2 | -0.3 | 0.2 | 0.4 | 0.5 | 0.8 | 1.9 | 0.6 | 0.5 | 0.1 | -0.1 | -0.0 | -0.1 | -0.1 | -0.3 | -0.4 | -0.4 | -0.2 | -0.2 | -0.3 |
| US12 | -0.2 | -0.3 | -0.4 | -0.3 | -0.2 | -0.3 | 0.1 | 0.0 | -0.0 | 0.2 | 0.6 | 1.6 | 0.8 | 0.7 | 0.3 | -0.1 | 0.0 | 0.0 | 0.0 | -0.2 | -0.3 | -0.6 | -0.3 | -0.2 |
| US13 | -0.2 | -0.4 | -0.3 | -0.3 | -0.2 | -0.2 | 0.0 | -0.0 | 0.0 | 0.1 | 0.5 | 0.8 | 2.0 | 1.1 | 0.5 | -0.1 | 0.0 | 0.1 | 0.1 | -0.1 | -0.2 | -0.4 | -0.3 | -0.2 |
| US14 | -0.3 | -0.3 | -0.3 | -0.6 | -0.6 | -0.3 | 0.2 | -0.0 | 0.1 | -0.0 | 0.1 | 0.7 | 1.1 | 2.5 | 1.0 | -0.0 | 0.1 | 0.1 | 0.2 | 0.1 | -0.1 | -0.7 | -0.6 | -0.4 |
| US15 | -0.2 | -0.1 | -0.3 | -0.6 | -0.7 | -0.4 | 0.2 | -0.1 | 0.3 | 0.1 | -0.1 | 0.3 | 0.5 | 1.0 | 2.9 | 0.2 | 0.1 | 0.1 | 0.1 | 0.1 | 0.1 | -0.5 | -0.4 | -0.4 |
| US16 | 0.1 | -0.0 | -0.0 | 0.0 | -0.1 | -0.2 | -0.2 | -0.2 | -0.2 | -0.0 | -0.0 | -0.1 | -0.1 | -0.0 | 0.2 | 1.2 | 0.4 | 0.2 | 0.0 | 0.1 | 0.2 | 0.2 | 0.1 | 0.0 |
| US17 | 0.1 | -0.1 | -0.1 | -0.1 | -0.2 | -0.3 | -0.3 | -0.2 | -0.3 | -0.2 | -0.1 | 0.0 | 0.0 | 0.1 | 0.1 | 0.4 | 0.9 | 0.4 | 0.3 | 0.1 | 0.1 | 0.1 | 0.2 | 0.2 |
| US18 | 0.1 | -0.0 | -0.1 | -0.2 | -0.3 | -0.3 | -0.3 | -0.2 | -0.3 | -0.3 | -0.1 | 0.0 | 0.1 | 0.1 | 0.1 | 0.2 | 0.4 | 0.8 | 0.4 | 0.2 | 0.1 | 0.1 | 0.1 | 0.2 |
| US19 | 0.2 | 0.0 | -0.1 | -0.3 | -0.4 | -0.4 | -0.4 | -0.3 | -0.4 | -0.4 | -0.3 | 0.0 | 0.1 | 0.2 | 0.1 | 0.0 | 0.3 | 0.4 | 1.3 | 0.4 | 0.2 | 0.1 | 0.1 | 0.2 |
| US20 | 0.2 | 0.1 | -0.0 | -0.1 | -0.3 | -0.3 | -0.4 | -0.3 | -0.2 | -0.3 | -0.4 | -0.2 | -0.1 | 0.1 | 0.1 | 0.1 | 0.1 | 0.2 | 0.4 | 1.3 | 0.3 | 0.3 | 0.2 | 0.1 |
| US21 | 0.2 | 0.1 | -0.0 | -0.1 | -0.2 | -0.2 | -0.3 | -0.3 | -0.2 | -0.2 | -0.4 | -0.3 | -0.2 | -0.1 | 0.1 | 0.2 | 0.1 | 0.1 | 0.2 | 0.3 | 0.8 | 0.5 | 0.4 | 0.3 |
| US22 | 0.4 | -0.0 | 0.3 | 0.6 | 0.2 | -0.2 | -0.8 | -0.5 | -0.6 | -0.4 | -0.2 | -0.6 | -0.4 | -0.7 | -0.5 | 0.2 | 0.1 | 0.1 | 0.1 | 0.3 | 0.5 | 2.5 | 0.9 | 0.5 |
| US23 | 0.4 | 0.1 | 0.2 | 0.3 | 0.2 | -0.2 | -0.4 | -0.2 | -0.5 | -0.4 | -0.2 | -0.3 | -0.3 | -0.6 | -0.4 | 0.1 | 0.2 | 0.1 | 0.1 | 0.2 | 0.4 | 0.9 | 1.7 | 0.7 |
| US24 | 0.4 | 0.2 | 0.1 | 0.1 | 0.1 | -0.1 | -0.4 | -0.2 | -0.5 | -0.4 | -0.3 | -0.2 | -0.2 | -0.4 | -0.4 | 0.0 | 0.2 | 0.2 | 0.2 | 0.1 | 0.3 | 0.5 | 0.7 | 1.3 |




### 1.0.5 Calculando norma das matrizes de covariância

[10]:
```
# Calculando a norma entre a matriz de covariancia e a matriz de covariancia de␣
↪referencia para cada metodo.
mcovars = [mcovar1, mcovar2, mcovar3, mcovar4]
C_ref_4 = np.cov(X_4.T)
C_ref_24 = np.cov(X_24.T)
normas_4 = []
normas_24 = []
print("Norma p/ 4 sensores.")
for i in range(4):
    C = mcovars[i](X_4.T)
```

```
    norma = np.linalg.norm(C - C_ref_4)
    normas_4.append(norma)
    print(f"Mcovar {i+1} | norma(C - C_ref): {norma}")

print("-" * 50)
print("Norma p/ 24 sensores.")
for i in range(4):
    C = mcovars[i](X_24.T)
    norma = np.linalg.norm(C - C_ref_24)
    normas_24.append(norma)
    print(f"Mcovar {i+1} | norma(C - C_ref): {norma}")
```

```
Norma p/ 4 sensores.
Mcovar 1 | norma(C - C_ref): 0.0001611620742082004
Mcovar 2 | norma(C - C_ref): 0.00016116207420584133
Mcovar 3 | norma(C - C_ref): 0.00016116207420160746
Mcovar 4 | norma(C - C_ref): 0.00016116207420574234
--------------------------------------------------
Norma p/ 24 sensores.
Mcovar 1 | norma(C - C_ref): 0.00212266396148315
Mcovar 2 | norma(C - C_ref): 0.002122663961456295
Mcovar 3 | norma(C - C_ref): 0.0021226639614547486
Mcovar 4 | norma(C - C_ref): 0.0021226639614562023
```

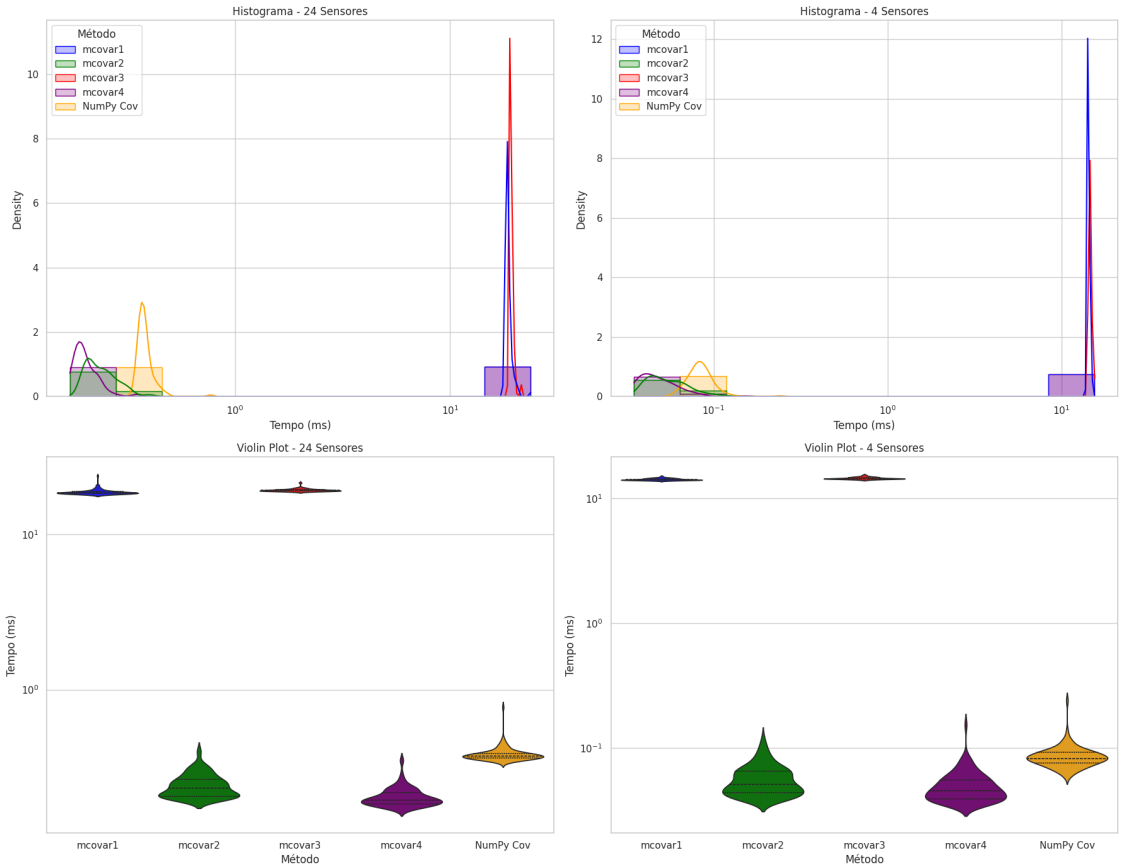
### 1.0.6 5. Comparativo de Desempenho (Lado a Lado)

Executamos o benchmark para ambos os datasets, agora incluindo os métodos externos `mcovar1`, `mcovar2`, e `mcovar4`. Note que `mcovar1` utiliza um loop explícito e pode ser significativamente mais lento.

[11]:
```
# 1. Executar Benchmarks
print("Executando benchmark para 24 sensores (100 rodadas)...")
results_24 = run_benchmark(X_24)

print("Executando benchmark para 4 sensores (100 rodadas)...")
results_4 = run_benchmark(X_4)

# 2. Preparar DataFrames
def prepare_df(results):
    data_list = []
    for method, times in results.items():
        for t in times:
            # Convertendo segundos para milissegundos
            data_list.append({'Tempo (ms)': t * 1000, 'Método': method})
    return pd.DataFrame(data_list)

df_24 = prepare_df(results_24)
df_4 = prepare_df(results_4)
```

```python
# 3. Plotagem Lado a Lado (2x2 Grid)
fig, axes = plt.subplots(2, 2, figsize=(18, 14))

# Paleta de Cores
color_palette = {
    'mcovar1': 'blue',
    'mcovar2': 'green',
    'mcovar3': 'red',
    'mcovar4': 'purple',
    'NumPy Cov': 'orange'
}

# Histograma - 24 Sensores
sns.histplot(data=df_24, x='Tempo (ms)', hue='Método', kde=True,␣
 ↪element="step", ax=axes[0, 0], palette=color_palette, log_scale=True,␣
 ↪stat="density")
axes[0, 0].set_title('Histograma - 24 Sensores')

# Histograma - 4 Sensores
sns.histplot(data=df_4, x='Tempo (ms)', hue='Método', kde=True, element="step",␣
 ↪ax=axes[0, 1], palette=color_palette, log_scale=True, stat="density")
axes[0, 1].set_title('Histograma - 4 Sensores')

# Violin Plot - 24 Sensores
sns.violinplot(data=df_24, x='Método', y='Tempo (ms)', inner="quartile",␣
 ↪ax=axes[1, 0], palette=color_palette, log_scale=True)
axes[1, 0].set_title('Violin Plot - 24 Sensores')

# Violin Plot - 4 Sensores
sns.violinplot(data=df_4, x='Método', y='Tempo (ms)', inner="quartile",␣
 ↪ax=axes[1, 1], palette=color_palette, log_scale=True)
axes[1, 1].set_title('Violin Plot - 4 Sensores')

plt.tight_layout()
plt.show()
```

```
Executando benchmark para 24 sensores (100 rodadas)…
Executando benchmark para 4 sensores (100 rodadas)…
```




[12]:
```
import pandas as pd
from IPython.display import display

def print_stats(results, title):
    df = pd.DataFrame([
        [method, np.mean(times) * 1000, np.std(times) * 1000]
        for method, times in results.items()
    ], columns=["Método", "Média (ms)", "Desvio (ms)"])
    print(f"\n{title}")
    display(df)

print_stats(results_24, "Resultados Estatísticos (24 Sensores)")
print_stats(results_4, "Resultados Estatísticos (4 Sensores)")
```

```
Resultados Estatísticos (24 Sensores)

      Método  Média (ms)  Desvio (ms)
0    mcovar1   18.683264     0.727337
1    mcovar2    0.242383     0.042793
2    mcovar3   19.246902     0.425369
```

```
3    mcovar4    0.204798     0.030930
4  NumPy Cov    0.385212     0.047112


Resultados Estatísticos (4 Sensores)

      Método  Média (ms)  Desvio (ms)
0    mcovar1   14.282194     0.254291
1    mcovar2    0.056090     0.016063
2    mcovar3   14.581124     0.283762
3    mcovar4    0.049709     0.015669
4  NumPy Cov    0.085934     0.020295
```

### 1.0.7 Testando estatisticas sobre os resultados

- Agora testaremos se o calculo de covariancia do numpy e os metodos implementados, entre si, são estatisticamente diferentes, estamos interessados especialmente se `mcovar1` e `mcovar3` são estatisticamente diferentes. Assim como `mcovar4`e `NumPy Cov`.

[13]:
```
from scipy.stats import mannwhitneyu
df_24_m1 = df_24[df_24['Método'] == 'mcovar1']
df_24_m2 = df_24[df_24['Método'] == 'mcovar2']
df_24_m3 = df_24[df_24['Método'] == 'mcovar3']
df_24_m4 = df_24[df_24['Método'] == 'mcovar4']
df_24_m5 = df_24[df_24['Método'] == 'NumPy Cov']
```

[14]:
```
stats, p = mannwhitneyu(df_24_m5['Tempo (ms)'], df_24_m4['Tempo (ms)'])
if p < 0.05:
    print("Existem diferenças significativas entre os métodos mcovar4 e Numpy␣
 ↪Cov")
else:
    print("Não existem diferenças significativas entre os métodos")
```

```
Existem diferenças significativas entre os métodos mcovar4 e Numpy Cov
```

[15]:
```
stats, p = mannwhitneyu(df_24_m2['Tempo (ms)'], df_24_m4['Tempo (ms)'])
if p < 0.05:
    print("Existem diferenças significativas entre os métodos mcovar2 e␣
 ↪mcovar4")
else:
    print("Não existem diferenças significativas entre os métodos")
```

```
Existem diferenças significativas entre os métodos mcovar2 e mcovar4
```

[16]:
```
stats, p = mannwhitneyu(df_24_m1['Tempo (ms)'], df_24_m3['Tempo (ms)'])
if p < 0.05:
    print("Existem diferenças significativas entre os métodos mcovar1 e␣
 ↪mcovar3")
else:
```

```
    print("Não existem diferenças significativas entre os métodos")
```

```
Existem diferenças significativas entre os métodos mcovar1 e mcovar3
```

## 1.1 Q3 - Avaliação da invertibilidade com base no metodo mais rapido.

[17]:
```
data_24_df.head()
```

[17]:
```
     US1    US2    US3    US4  US5    US6  US7    US8    US9   US10  …  \
0  0.438  0.498  3.625  3.645  5.0  2.918  5.0  2.351  2.332  2.643  …
1  0.438  0.498  3.625  3.648  5.0  2.918  5.0  2.637  2.332  2.649  …
2  0.438  0.498  3.625  3.629  5.0  2.918  5.0  2.637  2.334  2.643  …
3  0.437  0.501  3.625  3.626  5.0  2.918  5.0  2.353  2.334  2.642  …
4  0.438  0.498  3.626  3.629  5.0  2.918  5.0  2.640  2.334  2.639  …

    US16   US17   US18   US19   US20   US21   US22   US23   US24  \
0  0.593  0.502  0.493  0.504  0.445  0.431  0.444  0.440  0.429
1  0.592  0.502  0.493  0.504  0.449  0.431  0.444  0.443  0.429
2  0.593  0.502  0.493  0.504  0.449  0.431  0.444  0.446  0.429
3  0.593  0.502  0.493  0.504  0.449  0.431  0.444  0.444  0.429
4  0.592  0.502  0.493  0.504  0.449  0.431  0.444  0.441  0.429

               Class
0  Slight-Right-Turn
1  Slight-Right-Turn
2  Slight-Right-Turn
3  Slight-Right-Turn
4  Slight-Right-Turn

[5 rows x 25 columns]
```

[19]:
```
# Usando mcovar4 iremos estimar a matriz de covariancia de cada classe.
import numpy as np

def analyze_invertibility(df, label):
    print(f"Análise de Invertibilidade: {label}")
    X = df.drop(columns=['Class'])
    y = df['Class']

    # Matriz de Covariância Global
    cov_global = mcovar4(X.T)
    rank_g = np.linalg.matrix_rank(cov_global)
    cond_g = np.linalg.cond(cov_global)
    rcond_g = 1 / cond_g
    print(f"Global -> Posto: {rank_g}, Cond: {cond_g:.2e}, RCond: {rcond_g:.
 ↪2e}")
```

```
    # Matrizes de Covariância por Classe
    for c in np.unique(y):
        X_c = X[y == c]
        cov_c = mcovar4(X_c.T)
        rank_c = np.linalg.matrix_rank(cov_c)
        cond_c = np.linalg.cond(cov_c)
        rcond_c = 1 / cond_c
        print(f"Classe {c} -> Posto: {rank_c}, Cond: {cond_c:.2e}, RCond:␣
↪{rcond_c:.2e}")
    print("-" * 50)

analyze_invertibility(data_24_df, "data_24_df")
analyze_invertibility(data_4_df, "data_4_df")
```

```
Análise de Invertibilidade: data_24_df
Global -> Posto: 24, Cond: 2.05e+01, RCond: 4.87e-02
Classe Move-Forward -> Posto: 24, Cond: 5.35e+01, RCond: 1.87e-02
Classe Sharp-Right-Turn -> Posto: 24, Cond: 3.13e+01, RCond: 3.20e-02
Classe Slight-Left-Turn -> Posto: 24, Cond: 1.73e+02, RCond: 5.77e-03
Classe Slight-Right-Turn -> Posto: 24, Cond: 1.96e+02, RCond: 5.10e-03
--------------------------------------------------
Análise de Invertibilidade: data_4_df
Global -> Posto: 4, Cond: 7.57e+00, RCond: 1.32e-01
Classe Move-Forward -> Posto: 4, Cond: 6.43e+01, RCond: 1.55e-02
Classe Sharp-Right-Turn -> Posto: 4, Cond: 2.15e+02, RCond: 4.65e-03
Classe Slight-Left-Turn -> Posto: 4, Cond: 1.11e+01, RCond: 9.00e-02
Classe Slight-Right-Turn -> Posto: 4, Cond: 2.19e+02, RCond: 4.57e-03
--------------------------------------------------
```

### 1.1.1 As matrizes acimas são singulares, não possuem posto completo. Assim, não podem ser invertidas e iremos regularizar.

[21]:
```
def reg_matriz(df: pd.DataFrame) -> pd.DataFrame:
    """
    Adiciona regularização a uma matriz de covariância.
    """

    lamb = 0.1
    lambI = lamb * np.eye(df.shape[1])
    return df + lambI
```

[22]:
```
def inverter_reg(df, label):
    print(f"Análise de Invertibilidade: {label}")
    X = df.drop(columns=['Class'])
    y = df['Class']

    # Matriz de Covariância Global
```

```
    cov_global = mcovar4(X.T)
    cov_global = reg_matriz(cov_global)

    # Tentar inverter, se possivel, retornar mensagem de erro
    try:
        np.linalg.inv(cov_global)
        print("Global -> Invertível")
    except np.linalg.LinAlgError:
        print("Global -> Não Invertível")

    rank_g = np.linalg.matrix_rank(cov_global)
    cond_g = np.linalg.cond(cov_global)
    rcond_g = 1 / cond_g
    print(f"Global -> Posto: {rank_g}, Cond: {cond_g:.2e}, RCond: {rcond_g:.
↪2e}")

    # Matrizes de Covariância por Classe
    for c in np.unique(y):
        X_c = X[y == c]
        cov_c = mcovar4(X_c.T)

        cov_c = reg_matriz(cov_c)

        try:
            np.linalg.inv(cov_c)
            print(f"Classe {c} -> Invertível")
        except np.linalg.LinAlgError:
            print(f"Classe {c} -> Não Invertível")

        rank_c = np.linalg.matrix_rank(cov_c)
        cond_c = np.linalg.cond(cov_c)
        rcond_c = 1 / cond_c
        print(f"Classe {c} -> Posto: {rank_c}, Cond: {cond_c:.2e}, RCond:␣
↪{rcond_c:.2e}")
    print("-" * 50)

inverter_reg(data_24_df, "data_24_df")
inverter_reg(data_4_df, "data_4_df")
```

```
Análise de Invertibilidade: data_24_df
Global -> Invertível
Global -> Posto: 24, Cond: 1.64e+01, RCond: 6.09e-02
Classe Move-Forward -> Invertível
Classe Move-Forward -> Posto: 24, Cond: 3.18e+01, RCond: 3.15e-02
Classe Sharp-Right-Turn -> Invertível
Classe Sharp-Right-Turn -> Posto: 24, Cond: 2.35e+01, RCond: 4.26e-02
Classe Slight-Left-Turn -> Invertível
```

```
Classe Slight-Left-Turn -> Posto: 24, Cond: 3.10e+01, RCond: 3.23e-02
Classe Slight-Right-Turn -> Invertível
Classe Slight-Right-Turn -> Posto: 24, Cond: 6.05e+01, RCond: 1.65e-02
--------------------------------------------------
Análise de Invertibilidade: data_4_df
Global -> Invertível
Global -> Posto: 4, Cond: 4.22e+00, RCond: 2.37e-01
Classe Move-Forward -> Invertível
Classe Move-Forward -> Posto: 4, Cond: 6.04e+00, RCond: 1.66e-01
Classe Sharp-Right-Turn -> Invertível
Classe Sharp-Right-Turn -> Posto: 4, Cond: 1.17e+01, RCond: 8.54e-02
Classe Slight-Left-Turn -> Invertível
Classe Slight-Left-Turn -> Posto: 4, Cond: 3.34e+00, RCond: 3.00e-01
Classe Slight-Right-Turn -> Invertível
Classe Slight-Right-Turn -> Posto: 4, Cond: 3.84e+00, RCond: 2.60e-01
--------------------------------------------------
```